

Kafka Message Duplication Handling SOP

Objective

Identify and reduce duplicate Kafka message processing caused by producer retries, consumer retries, or offset commit behavior.

Scope

Applicable to Kafka-based applications, streaming platforms, backend services, data engineering pipelines, DevOps teams, and platform teams supporting production or synthetic incident workflows.

Pre-requisites

1. Access to Kafka cluster, namespace or broker host.
2. Access to monitoring dashboards such as Grafana, Prometheus, or cloud monitoring.
3. Kafka CLI tools: kafka-topics, kafka-consumer-groups, kafka-console-producer, and kafka-console-consumer.
4. Permission to view application logs, broker logs, and deployment status.
5. Escalation contact for DevOps, platform, and application owner teams.

Incident Metadata

Field	Value
Ticket ID	T019
Issue	Kafka message duplication
Category	Streaming
Service	Kafka
Severity	Medium
Status	Resolved
Description	Duplicate messages received
Expected Resolution	Enable idempotent producer settings

Related Log Evidence

No direct matching log entry was present in the provided logs CSV. Use the ticket description, application logs, and monitoring dashboards to validate this incident in runtime.

Procedure

1. Confirm duplicate message symptom by checking message key, event ID, timestamp, and consumer output.
2. Identify whether duplication is producer-side, broker-side, or consumer-side.
3. Verify producer retry configuration and idempotence settings.
4. Check whether consumers commit offsets before or after processing.
5. Review consumer crashes or rebalances around duplicate processing time.
6. Enable or validate idempotent producer settings where supported.
7. Add consumer-side idempotency using event ID or business key if required.
8. Monitor duplicate rate and update ticket with root cause and prevention action.

Common Commands

```
kafka-topics.sh --bootstrap-server <broker>:9092 --describe
kafka-consumer-groups.sh --bootstrap-server <broker>:9092 --describe --group <group>
kafka-broker-api-versions.sh --bootstrap-server <broker>:9092
kubectl get pods -n <namespace> | grep kafka
```

```
kubect1 logs <kafka-pod> -n <namespace> --tail=100
```

Troubleshooting Matrix

Issue	Possible Cause	Resolution
Duplicate events	Producer retry without idempotence	Enable idempotent producer
Repeated processing	Offset commit after crash	Fix commit strategy
Duplicates after rebalance	Consumer group instability	Tune heartbeat and poll interval
Same business record repeated	No idempotency key	Use event ID based deduplication

Best Practices

1. Use message keys and event IDs for traceability.
2. Design consumers to be idempotent.
3. Avoid committing offsets before successful processing unless safe.
4. Track duplicate counts in observability metrics.

References

Apache Kafka Docs | Internal Logs CSV | Tickets CSV | Monitoring Dashboards | Kafka CLI Runbook